Presentation Script — Click by Click

AetherOS + SpillGuard · Varad Patil (24BIT0175) · for Prof. Harshita Patel

Every button name and screen below was checked by clicking through the real app on this laptop. Each step tells you exactly what to **click**, what you will **see**, what to **say** to Ma'am, and what she might **ask**.

Label	Meaning
CLICK	Exactly what to press or type. Button names are in **boxes**.
SEE	What appears on the screen, so you know it worked.
SAY	What to explain, in your own voice. Speak slowly; point at the screen.
IF ASKED	Likely follow-up questions and short answers.
TIP	Practical advice so nothing goes wrong.

Running order and timing (about 20 minutes)

Part	What you show	Time
0	Preparation (before Ma'am arrives)	10 min before
1	Boot, login, desktop tour	1.5 min
2	CPU scheduling — Task Manager (Round Robin, MLFQ, two cores, work stealing)	4 min
3	Memory — paging, page faults, LRU / FIFO / CLOCK	2.5 min
4	Deadlock — detection, recovery, Banker's algorithm (textbook and on real VRAM)	3 min
5	Terminal — commands and the protection fault (#GP)	1.5 min
6	File system — power cut and fsck recovery	1.5 min
7	eBPF Studio, Test Suite (quick)	1 min
8	SpillGuard live — real GPU memory of this laptop	2 min
9	Spill demonstration — replay of a real spill	2 min
10	Research results — the evidence desk	3 min
11	Conclusion	0.5 min

Part 0 — Preparation (do this before Ma'am comes)

PREP 1 — Charge and free memory

DO Plug in the charger. Close Chrome/Edge tabs you don't need, WhatsApp, games, Epic Games Launcher, and anything heavy.

WHY Your laptop has 16 GB RAM. When I tested, Windows' memory commit was at 90 %, so SpillGuard showed a **RAM_PRESSURE** event and the "Will it fit?" check said UNSAFE because of RAM. With apps closed it shows the normal healthy picture.

PREP 2 — Start the project

DO Open the project folder `aetheros` and double-click `START_DEMO.bat`. A black window opens and stays open — **do not close it** during the presentation.

SEE The browser opens by itself at `http://127.0.0.1:8765/…` with the AetherOS boot screen.

TIP Always use the browser tab that `START_DEMO.bat` opens. That link carries a secret one-time token; if you type the address yourself, the Live GPU data will not connect (SpillGuard will say "Waiting for the agent…"). If that happens, close the black window and double-click `START_DEMO.bat` again.

TIP Press **F11** in the browser for full screen — it looks much better on a projector.

PREP 3 — Do one practice run, then refresh

DO Go through Parts 1–4 quickly once. Then press **F5** to reload so Ma'am sees a fresh boot. (Reloading resets the simulated kernel to tick 0.)

Window buttons (top-right of every AetherOS window, left to right): **green = minimize**, **yellow = maximize**, **red = close**. Maximize each app when you present it, and close it with red when you move on — otherwise windows pile up on top of each other. Closing a window does **not** stop the kernel: processes keep running in the background.

Part 1 — Boot, login and desktop tour

STEP 1 — Boot screen

SEE "AetherOS System 7 — Deterministic Microkernel" with a progress bar: *Initializing AetherOS Microkernel... Mounting Inode Virtual File System... Initializing Dual-Core Schedulers (MLFQ/RR)... Allocating 32-Frame Virtual Memory Matrix... Ready.*

SAY *"Ma'am, my project has two parts. AetherOS is an operating-system simulator I built that runs in the browser — you can see it booting: it mounts a file system, starts two CPU schedulers and allocates 32 memory frames. The second part, SpillGuard, is a real monitoring agent for the GPU memory of this laptop. The kernel here is a simulation for teaching; SpillGuard measures real hardware."*

STEP 2 — Login ("password")

CLICK Click the **Enter Password** box, type `Varad`, press **Enter**.

SEE The name above the box changes to "Varad" as you type. The hint bubble says "It's open. Just press Enter." After Enter, the desktop appears with a welcome popup.

SAY *"The login is open for the demo — whatever name I type personalises the session; there is no real password check."*

IF ASKED "Is there authentication?" — *"Not for the simulator. For the real SpillGuard agent there is security: it only listens on 127.0.0.1, needs a random session token, and checks the Host and Origin headers."*

STEP 3 — Welcome popup

SEE "Aethor OS is an interactive desktop inspired by classic Mac OS... New: SpillGuard shows your real GPU memory live, and catches the moment it spills into system RAM. Built by Varad."

CLICK The small red **x** at the popup's top-right corner.

STEP 4 — Desktop tour (just point, don't open yet)

SEE **Left icons:** ReadMe.txt, Calculator, **Dash's Journal** (= the Task Manager / CPU scheduler), **archive** (= Finder, the file system), **eBPF Studio**, **SpillGuard**.
Right: My Socials, Community. **Bottom dock:** Finder, Internet, Music, Messages, **Terminal** (the `>_` icon), Trash.
Top menu bar: logo menu, File, Edit, View, **Kernel**, **Diagnostic**. Bottom-right: the bird **Zephyr**, which gives tips.

SAY *"This is the desktop. The OS features are in these apps and menus: the Kernel menu controls the CPU clock and scheduler, the Diagnostic menu opens the test suite, deadlock engine, memory map and challenges, and the Edit menu can inject faults — a deadlock, a power cut or a protection fault. The bird in the corner gives tips and warns when GPU memory spills."*

CLICK (Optional) Click **Kernel** in the menu bar to show: *Pause Clock, Step Single Tick, Scheduler: MLFQ ✓, ROUND_ROBIN, SRTF, FCFS, PRIORITY*. Press **Esc** to close it.

SAY *"The kernel runs on a clock: every 800 milliseconds is one tick, one unit of CPU time. I can pause it or step one tick at a time to explain exactly what happens."*

Part 2 — CPU scheduling (Task Manager)

STEP 5 — Open the Task Manager

CLICK Desktop icon **Dash's Journal**. Then the **yellow** button at the window's top-right to maximize.

SEE Title "Task Manager - Dual-Core SMP Scheduler". Top bar: **Pause Clock**, "Tick: #...", "Policy: MLFQ (Multi-Level Feedback)", Speed slider. Two boxes "CPU Core 0 (Primary)" and "CPU Core 1 (Secondary)" each with *Load %, CURRENT, RUNQUEUE, STOLEN*. Two coloured timelines. "Spawn Process" form. At the bottom the "Process Control Block (PCB) Multi-Core Register Table".

SAY *"This is the scheduler on two simulated CPU cores. The table at the bottom is the Process Control Block: for each process its PID, state, core affinity, the last core it ran on, remaining burst over total burst, waiting time, and its MLFQ queue level. At boot four system processes were created — kernel_daemon, window_manager, audio_server, network_sync — and you can see they are already TERMINATED because they finished their CPU bursts. The timelines are Gantt charts of the last 30 ticks, one row per core; grey means idle."*

IF ASKED "What is a PCB?" — *"The record the OS keeps per process: ID, state, priority, CPU-time used and remaining, memory frames, locks held and so on."*

STEP 6 — Set up a Round Robin experiment

CLICK
1. **Pause Clock** (it changes to "Resume Clock").
2. Policy dropdown → **Round Robin (Preemptive)**.
3. In Spawn Process: click the name box, delete the text, type `A`. Click the **Burst** box, type `8`. Click **+ Dispatch Process**.
4. Name box → type `B` → **+ Dispatch Process**. Name box → `C` → **+ Dispatch Process**.

TIP After each dispatch the name box changes to something like "task_716" — type the new name every time. Burst stays 8. Burst can be at most 50.

SAY *"I paused the clock and created three processes A, B and C, each needing 8 ticks of CPU. In Round Robin each process gets a time quantum of 4 ticks, then goes to the back of the queue. New processes are placed on the core with the shorter run queue — that is load balancing."*

STEP 7 — Run it and read the Gantt chart

CLICK **Resume Clock**. Wait about 15–20 seconds, pointing at the two timelines.

SEE (Real result from my test, PIDs will differ:) **CPU 0**: A's PID four times, then C's PID four times, then A again. **CPU 1**: B runs alone. When B finishes, CPU 1's **STOLEN** becomes **+1** and C's PID appears on CPU 1.

SAY *"Look at Core 0: A ran exactly 4 ticks — its quantum — then it was preempted and C ran for 4 ticks, then A came back. That is Round Robin. B was alone on Core 1. And when Core 1 became empty it stole work from Core 0's queue: the STOLEN counter became plus one. That is work stealing — an idle core takes half of the other core's waiting tasks from the tail of its queue, which is how real multi-core schedulers keep all cores busy."*

IF ASKED "What if the quantum is very large?" — *"Round Robin becomes FCFS; too small and we waste time on context switches."* "How do you count context switches?" — *"Every tick where a core runs a different PID than in the previous tick."* "What is affinity?" — *"Pinning a process to one core. In the PCB table I can click the AFFINITY cell to toggle it, or choose Core 0 Only / Core 1 Only when spawning."*

STEP 8 — MLFQ (the default algorithm)

CLICK Policy → **MLFQ (Multi-Level Feedback)**. Name `longjob`, Burst `50`, **+ Dispatch Process**. Scroll the PCB table down to the new row and watch its **LEVEL** column for ~20 seconds.

SEE LEVEL goes **Q0 → Q1 → Q2** as the job keeps using its full time slice.

SAY *"MLFQ has three queues with quanta 4, 8 and 16 ticks. A new job starts in Q0, the highest priority. If it uses its whole quantum, it is CPU-bound, so it is demoted to Q1, then Q2. Short interactive jobs stay on top. To prevent starvation, every 40 ticks all jobs are boosted back to Q0 — that is aging. MLFQ approximates Shortest-Job-First without knowing burst times in advance."*

IF ASKED "Other algorithms?" — *"FCFS picks the earliest arrival, SJF the shortest total burst, SRTF the shortest remaining time, Priority the smallest priority number. All six are in this dropdown and in the Kernel menu."*

Part 3 — Memory: paging and page replacement

STEP 9 — Fill the memory (still in Task Manager)

CLICK — **Pause Clock**. Burst box → `50`. Click **+ Dispatch Process** **9 times** (any names are fine). Then **Resume Clock**. Close the Task Manager with the **red** button.

SAY — *"Each process I create needs 16 KB, which is 4 pages of 4 KB. The machine has only 32 frames, so 8 processes fill the memory and the 9th must cause page faults."*

STEP 10 — Open the Memory Map

CLICK — Menu bar **Diagnostic** → **Virtual Memory Map**. Maximize (yellow).

SEE — "Virtual Memory & Paging Unit", Eviction Policy dropdown (LRU), four boxes: *Total Physical RAM 128 KB (32 Frames)*, *Allocated Frames 32 Used / 0 Free*, *Memory Load 100%*, *Total Page Faults* (a number above 0 — I got 6). Below: a 4×8 grid of frames F0–F31, coloured by owner process with its PID.

SAY — *"This is the physical frame map — 32 frames of 4 KB. Every colour is one process. Memory is now 100% full, so when the last process arrived there was no free frame: those were page faults, and the replacement policy chose victim frames from other processes. The policy is LRU, Least Recently Used: it evicts the frame whose last access is oldest."*

STEP 11 — Inspect a frame

CLICK — Click frame **F0**.

SEE — "Frame #0 Inspector — Status: Allocated, Owning PID: …, Virtual Page: 0, Last Access Tick: #…"

SAY — *"Each frame records which process and which virtual page it holds, and the last tick it was accessed. Every tick a process runs, its frames are touched — that timestamp is what LRU uses, and the reference bit is what CLOCK uses."*

STEP 12 — Change the policy

CLICK — Eviction Policy dropdown → show the four options: *LRU, FIFO, Clock (Second Chance), Optimal (needs future refs: runs LRU)*. Choose **Clock (Second Chance)**.

SAY — *"FIFO evicts the page loaded first; it can suffer Belady's anomaly. CLOCK is what real operating systems use: a hand sweeps the frames in a circle; if a frame's reference bit is 1 it gets a second chance — the bit is cleared — and if it is 0 it is evicted. Optimal would evict the page used farthest in the future, but no real system knows the future, so I label it honestly: it falls back to LRU."*

TIP — The new policy applies to the *next* eviction. To show it, open Task Manager and dispatch one more process, then come back — Total Page Faults increases by 4.

STEP 13 — The real-GPU "twin view"

CLICK Scroll down inside the Memory Map.

SEE "Twin view · real GPU memory (live)": a bar "VRAM = physical frames · 5.86 GB on NVIDIA GeForce RTX 3050 6GB Laptop GPU" and below it "Shared system RAM = backing store · spilled pages live here", then per-process bars.

SAY *"This is the bridge to my second part. On a real GPU the same idea exists: the VRAM on the graphics card is like physical frames, and system RAM acts as the backing store. When VRAM is full, Windows moves GPU data into system RAM — exactly like paging. These bars are the real memory of this laptop's GPU right now."*

CLICK Close the Memory Map (red).

Part 4 — Deadlock and Banker's algorithm

STEP 14 — Open the Deadlock Engine

CLICK **Diagnostic** → **Open Deadlock Engine**. Maximize (yellow).

SEE Green bar "SYSTEM HEALTHY (No Circular Wait) — All process resource allocation matrices verified acyclic", a red **Inject Deadlock** button, the Banker's table, and "Banker's algorithm on real VRAM (live)" below.

SAY *"A deadlock needs four conditions at once: mutual exclusion, hold-and-wait, no preemption and circular wait. The engine checks continuously for a circular wait."*

STEP 15 — Inject a deadlock

CLICK **Inject Deadlock**.

SEE Red bar "DEADLOCK DETECTED (Tarjan SCC Cycle) — Cyclic dependency found among PIDs: [x, y]" and "PREEMPT OFFENDING CYCLE PROCESSES:" with buttons **Preempt PID #x** **Preempt PID #y**.

SAY *"I created two threads. Alpha holds the GPU mutex and waits for the disk mutex; Beta holds the disk mutex and waits for the GPU mutex. The engine builds a wait-for graph — an arrow from a process to the process holding what it waits for — and runs a depth-first search. If the search reaches a node that is still on its recursion stack, there is a cycle: deadlock. It found the cycle between these two PIDs."*

IF ASKED "It says Tarjan — is that what you implemented?" — *"The detection is DFS-based cycle detection with a recursion stack (back edge = cycle). Tarjan's strongly-connected-components algorithm is also DFS-based; my implementation uses the simpler back-edge method, which is O(V+E)."* "How did you create the deadlock?" — *"Edit menu → Simulate Deadlock does the same thing."*

STEP 16 — Recover

CLICK **Preempt PID #x** (the first one).

SEE The bar turns green again: "SYSTEM HEALTHY (No Circular Wait)".

SAY *"Recovery by preemption: terminating one process on the cycle releases its lock, the circular wait is broken, and the other can continue."*

STEP 17 — Banker's algorithm (avoidance)

CLICK **Compute Safe Sequence**.

SEE "Safe State Discovered! Safe Execution Sequence: P1 → P3 → P4 → P0 → P2".

SAY *"Detection finds a deadlock after it happens; Banker's algorithm avoids it. Each process declares its maximum need. Need equals Max minus Allocation. Available is 3, 3, 2. P1 needs 1, 2, 2, which fits, so it can finish and return what it holds — work becomes 5, 3, 2. Then P3, P4, P0 and P2 can finish. Because an order exists in which everyone finishes, the state is safe."*

IF ASKED "Is unsafe the same as deadlock?" — *"No. Unsafe means a deadlock may happen; safe guarantees it cannot."*

STEP 18 — Banker's on REAL GPU memory

CLICK Scroll down to "Banker's algorithm on real VRAM (live)". Drag the slider "Run Safety Check for a new launch claiming … GB" to the right, to about **5.5 GB**.

SEE A table of real processes with Allocation / Max / Need in MB, "NEW LAUNCH" at the bottom, and two boxes: **Textbook Banker's: SAFE** and **Long-lived processes (SpillGuard): DENY** — e.g. "needs 5.5 GB VRAM, 4.1 GB free; pythonw.exe holds 1.2 GB".

SAY *"Here I use the same Banker's code on the real VRAM of this laptop. Allocation is what each real process holds now, Max is its peak times 1.1, and the new program — say an AI model — claims 5.5 GB. The textbook algorithm says SAFE because it assumes every process eventually finishes and returns its memory. But real apps like a browser or a game never finish, so SpillGuard reserves their remaining need and says DENY — launching this would push memory into overflow. That disagreement is one of my findings: textbook Banker's is too optimistic for long-running programs."*

TIP Process names depend on what is running on the laptop. Move the slider left to about 3 GB to show both boxes say SAFE / ADMIT. Close the Deadlock window (red) when done.

Part 5 — Terminal and the protection fault

STEP 19 — Open the terminal

CLICK Dock icon `>_` (Terminal). Click inside the input line.

SEE "aethersh v2.4 - Aether Shell CLI", prompt `varad@aether:/user$`.

STEP 20 — Process list

CLICK Type `ps` and press Enter.

SEE Table: PID, NAME, STATE, PRIO, CORE, BURST, REMAIN, MEM, and "[SMP LOAD] Core 0: ...% (... stolen) | Core 1: ...%".

SAY *"Like ps on Linux — every process with its state, priority, core affinity, burst and memory. The shell talks directly to the same kernel the windows show."*

STEP 21 — General Protection Fault

CLICK Type `gpf` and press Enter.

SEE "[GENERAL PROTECTION FAULT] Trapped invalid memory access! PID: ... Illegal Address: 0x0020 (Access Ring: RING_3) EIP ... ESP ... CR3 ... EAX ... CS 0x0073 EFLAGS ... Process halted and placed in WAITING state."

SAY *"CPUs have privilege rings: ring 0 is the kernel, ring 3 is user programs. In my address space 0x0000 to 0x3FFF is kernel memory. A user process tried to read address 0x0020, which is inside the kernel, so the memory manager trapped it as a General Protection Fault, stopped the process and printed a register dump: EIP is the faulting instruction, ESP the stack pointer, CR3 the page-table base, and CS 0x73 shows it was user-mode code."*

IF ASKED "Are those register values real?" — *"They are generated for illustration; the mechanism — check the ring and the address range, then trap — is the real concept."*

STEP 22 — Other commands (pick 2–3)

CLICK `memstat` (RAM and page faults) · `sched SRTF` (switch scheduler) · `bpf` (probe maps) · `spillguard status` (real GPU table) · `help` (full list). Type `clear` to clean the screen. Close the terminal (red).

SAY *"spillguard status prints the real VRAM, shared memory and RAM of this laptop per process — the same data as the SpillGuard window, inside my OS shell."*

Part 6 — File system: power cut and recovery

STEP 23 — Open Finder

CLICK Desktop icon **archive**.

SEE "Finder - Crash-Consistent WAL Inodes". Left: Root (/), User space, Config (etc); a **Superblock** box: *CLEAN, Mount: online, Inodes: 5 / 64, WAL entries: 0*, **View WAL ring**; "Crash consistency": **Power cut**, **Run fsck**. Files readme.txt and manifest.json.

SAY *"Each file is an inode — a record with its name, size, permissions and parent folder. The superblock describes the whole disk: clean or dirty, mounted or not."*

STEP 24 — Create a file (journaled)

CLICK Click the box "new_file.txt", type `notes.txt`, click **+ Create**.

SEE notes.txt appears; Inodes 6 / 64; WAL entries 1.

SAY *"Every change is first written to a journal — write-ahead logging. The entry goes PENDING, then COMMITTED, then CHECKPOINTED when it is applied to the inode table."*

STEP 25 — Power cut

CLICK **Power cut**, then **View WAL ring**.

SEE Red bar "Crash detected: dirty WAL blocks", "Disk unmounted after a power cut", Superblock **DIRTY**, Mount: unmounted. The WAL table shows #100 CREATE /user/notes.txt **CHECKPOINTED** and #101 WRITE /user/dirty_buffer.tmp **PENDING**.

SAY *"I simulated a sudden power failure in the middle of a write. The disk is now dirty and unmounted, and the journal has one torn write that never committed."*

STEP 26 — fsck recovery

CLICK **Run fsck recovery**.

SEE Dialog "fsck: crash-consistency recovery — Journal replayed": Phase 1 scanning… "Found uncommitted torn Txn #101 (/user/dirty_buffer.tmp). Rolling back dirty transaction." Phase 2 "1 torn txn rolled back, 0 replayed… Superblock marked CLEAN. Remounted VFS."

SAY *"fsck reads the journal: anything committed but not yet applied is replayed; anything still pending is rolled back. So the file system is never left half-written — notes.txt survived, the torn write disappeared. This is how ext4 and NTFS journaling work."*

CLICK **Done (remount)**, then close Finder (red).

Part 7 — eBPF Studio and the Test Suite (quick)

STEP 27 — eBPF Studio

CLICK Desktop icon **eBPF Studio**.

SEE Left: "Compile & attach a probe" form; "Attached probes (3)": sched_switch_counter (SCHED_SWITCH), mm_page_fault_tracer (PAGE_FAULT), vfs probe — each with an ON switch. Right: "Map: sched_switch_counter BPF_MAP_TYPE_HASH" with hits per PID, "Ring buffer stream" of events, and "Real host events · SpillGuard (live)".

SAY *"On Linux, eBPF lets small safe programs attach to kernel events and count them in maps. I simulate that: one probe counts context switches per process, one counts page faults. And the bottom box shows real events from SpillGuard on this laptop — spills and RAM pressure — in the same tracing view."*

IF ASKED "Does it really compile bytecode?" — *"No, the probe engine is simulated: the script is a label and the kernel dispatches events to matching probes. The real-host events panel is real."*

STEP 28 — Kernel Test Suite

CLICK Close eBPF Studio. **Diagnostic** → **Run Kernel Test Suite**. Click **Run all tests**.

SEE "Kernel verification suite — 13 / 13 all passing": FCFS, Round Robin ("Core 0 ran A A B B…"), SRTF, MLFQ aging, LRU, FIFO, Banker's unsafe state, cycle detection, inodes, terminal integration, state export/import. Tabs: Event journal, Disk snapshot.

SAY *"The kernel tests itself inside the app. Outside, I also have 66 automated front-end tests and 149 Python tests for the agent, all passing. While building it I found and fixed a real scheduler bug: a running process was skipped forever, so the quantum never applied; this test now proves Round Robin works."*

CLICK Close it (red).

Part 8 — SpillGuard live: the real GPU

STEP 29 — Explain the problem first (no click)

SAY *"Now the real part. A GPU has its own fast memory, VRAM — this laptop has 6 GB. If programs together need more than that, Windows and the NVIDIA driver don't show an error; they silently put the extra data in normal system RAM, reached over the PCIe bus. I measured that VRAM gives about 163 GB per second but data in system RAM only 13 GB per second — about 12 times slower. The program just becomes slow and nobody knows why. This is called a spill. Even CUDA's own 'how much is free' function said 0 MB free while allocations kept succeeding, so programs cannot see it. SpillGuard detects it from outside."*

STEP 30 — Open SpillGuard

CLICK Desktop icon **SpillGuard**. Maximize (yellow).

SEE Header: "NVIDIA GeForce RTX 3050 6GB Laptop GPU · driver 581.83", buttons **Simulated** **Live** **Replay**, and a green "● agent 127.0.0.1:8765". Green banner "No heuristic alert · …% of VRAM in use". Three gauges: **Dedicated VRAM** (with OS budget and 92 % marks), **Shared GPU memory** (cap = ½ RAM), **RAM commit**. A "Last … s" chart. "Per-process memory tiers". "Admission check (Banker's)". "Events". Then the evidence desk further down.

SAY *"This is live data from the Python agent running on this laptop. Once per second it reads Windows' own GPU performance counters — the same ones Task Manager uses — for every process: how much is in VRAM (blue) and how much is in shared system memory (red if it is a spill). The first gauge is VRAM with two marks: the budget Windows suggests and my 92 % alarm level. The second is shared GPU memory, whose limit is half of RAM. The third is RAM commit — if the whole computer runs out of RAM that is a different problem, which I label type C."*

IF ASKED "Where do the numbers come from?" — *"Windows PDH counters 'GPU Process Memory — Dedicated and Shared Usage', DXGI for the physical VRAM size, NVIDIA's NVML as a cross-check, psutil for RAM, and Ollama's API for AI models. No admin rights needed."*

STEP 31 — Will this AI model fit? (Banker's admission)

CLICK In "Admission check (Banker's)": model **qwen2.5:3b-instruct**, **ctx 8192**, **f16** → **Will it fit?** (takes a few seconds).

SEE "weights 1.79 GB · KV 288 MB · buffers 346 MB · CUDA 235 MB = 2.64 GB" and a verdict (SAFE / UNSAFE with a reason).

SAY *"Before launching an AI model SpillGuard predicts its GPU memory: weights, plus the KV cache — the memory that stores past tokens, which grows with context length — plus buffers and the CUDA context. I calibrated this on this laptop; the average error was 46 MB, while Ollama's own estimate was off by about 900 MB. Then it runs Banker's safety check with VRAM and RAM as the resources."*

TIP If it says "UNSAFE — needs … commit, … free", that is about **RAM**, not VRAM: say *"it refuses because this laptop's system RAM is nearly full right now — exactly the type C protection."* Try **q4_0** or a smaller ctx to show the estimate shrink.

Part 9 — Spill demonstration (replay of a real spill)

Why a replay? Causing a real spill on stage fills 6 GB of VRAM and a lot of RAM and could freeze the laptop. The replay plays a trace that was **recorded on this same laptop** during a real spill — exactly the same frames the live view would show.

STEP 32 — Start the replay

CLICK Scroll to the top of SpillGuard. Click **Replay**, then **Watch the demo spill**.

SEE A player bar: **Pause**, a seek slider, a speed box (4×), and "W1 spill stress · RTX 3050 6 GB…". VRAM rises steadily as python.exe allocates memory.

SAY *"This is a recording of my stress test: a program allocates GPU memory in 256 MB blocks. Watch the blue VRAM bar and the chart climb toward the dashed line, the physical VRAM limit."*

TIP At 4× the spill comes after about 50 seconds. To save time, set speed to **16×** or drag the seek slider to about **60 %** of its length.

STEP 33 — The spill moment

SEE The banner turns **red**: "Spill heuristic triggered: python.exe (960 MB, 14%)", and a line "Growth heuristic (correlation; cause unverified): python.exe grew by 3.31 GB in the 30 s before the alert." Dedicated VRAM 5.81 / 5.86 GB (red). Shared GPU memory jumps to ~1 GB. python.exe's bar gets a **red** segment "+960 MB". The bird Zephyr announces the spill.

CLICK **Pause** right here so Ma'am can read it.

SAY *"Here VRAM is at its physical limit and the program's shared memory grew by 960 MB — that part now lives in system RAM. My detector fires when three things hold for two seconds in a row: the GPU is at least 92 % full, the process's shared memory is more than 128 MB above its own normal baseline, and it is still rising. It also names the culprit — the process whose VRAM grew most in the 30 seconds before. Here python.exe caused its own spill. Against independent ground truth it caught 3 out of 3 spills, with zero false alarms, within about one second."*

IF ASKED "Why a baseline?" — *"Browsers and the Windows compositor always keep some shared memory; comparing each process with its own minimum avoids false alarms."* "Why 92 % and not the budget?" — *"I measured that under pressure the driver fills VRAM to 5954 of 6002 MB, far beyond the 5234 MB budget, so physical capacity is the real limit."*

CLICK (Optional) Scroll to **Events** to show SPILL_START. Press Play to let it finish and show SPILL_END.

Part 10 — Research results (evidence desk)

STEP 34 — Set the research question (no click)

SAY *"Detecting a spill is not enough — I wanted to prove how much it actually hurts a real application and what fixes it. So I ran controlled experiments with a real local AI model, Qwen 2.5 3B in Ollama. Each trial has three stages: baseline — the model alone; pressure — the same work with a second program, the co-tenant, holding GPU memory; and recovery — alone again. Same prompt, same 128 tokens, temperature zero, so the output must be identical. A trial counts as FAIL — the application failed to keep its speed — only if it becomes at least 2 times slower and recovers afterwards. Anything doubtful is INCONCLUSIVE, never hidden."*

STEP 35 — Open the headline chart

CLICK Scroll down inside SpillGuard to "SpillGuard Repro · evidence desk". The headline "Dose-response + intervention + remedies (all in one)" is already open. Scroll to the chart.

SEE Four boxes "1 Reproduce · 2 Witness · 3 Intervene · 4 Compare"; a chart of slowdown vs "planned overflow beyond physical VRAM (MiB)" with a "VRAM full" line and a "2× failure threshold" line; red dots at +256 and +512, black dots at −512 and 0; a square for the idle co-tenant.

SAY *"I varied how far the total memory demand goes past the 6 GB. At minus 512 MB — everything fits — it's only 1.16 times slower. Exactly at capacity, 1.21. But just 256 MB over capacity, decoding becomes 2.36 times slower — from 14 to 33 milliseconds per token. At 512 over it's the same, 2.33. So it is a cliff, not a slope: the moment you cross VRAM capacity, performance falls off."*

STEP 36 — Every trial is shown

CLICK Scroll to the table "Every trial (invalid ones included)".

SAY *"Every trial is listed, with baseline, pressure and recovery times. All failures recovered fully when the pressure was removed. Even the inconclusive ones are kept — two idle trials were stopped by my safety guard because the laptop's RAM commit reached 90 %."*

STEP 37 — Same counters, opposite outcome

CLICK Point at the **idle** rows (square on the chart, 1.03×) and the paragraph "Can memory counters tell harmful from benign pressure?".

SAY *"This is my most interesting finding. I repeated +256 MB with the co-tenant holding exactly the same memory but sitting idle. Windows' memory counters were identical in both cases — same VRAM, same shared memory — yet the active one caused a 2.4 times slowdown and the idle one only 1.03. So counters alone cannot tell harmful from harmless pressure. On these trials my own detector caught all 4 real slowdowns but also raised 3 alarms on harmless runs. The solution is an intervention: change one thing and measure."*

IF ASKED "Why does activity matter?" — *"My working hypothesis is that an active co-tenant makes Windows keep moving pages back and forth between VRAM and RAM — churn — which the counters don't capture. It is not yet proven; it needs ETW/PIX residency tracing, which is future work."*

STEP 38 — Which fix works?

CLICK Scroll to "Which fix works, and which disrupts least? (+256 MiB witness)".

SEE none 2.35× (reproduced again) · release whole co-tenant 1.02× · run it after the target 1.00× · release half 1.10× · release one eighth 1.19× · **keep every byte, stop touching it 1.01×**.

SAY *"First the failure was re-validated from scratch — it reproduced again, so it is repeatable: 6 out of 6 trials across two runs. Then each fix: freeing all memory, running the other program later, freeing half, or only one eighth — all work. The best one is the least disruptive: the co-tenant keeps all its memory, it just pauses its activity, and the AI model is back to 1.01. That is a practical remedy."*

STEP 39 — (Optional) Open one real experiment and one real GPU trace

CLICK Scroll up to the library list. Under "Dose-response runs" click **+256 MiB over capacity — reproduced witness**.

SEE "Slowdown reproduced · live measurement · Run complete", a "Matched block" selector and a table Baseline / Pressure / Recovery with times.

CLICK Expand "Live GPU traces of the witness (replay)" → **▶ +256 MiB trial 1 · pressure**. A green note says it was loaded into Replay. Scroll to the very top of SpillGuard and press Play.

SAY *"This is the actual GPU memory recording from that research trial: llama-server is the AI model and python is the co-tenant. Every run is saved with a SHA-256 hash of every file, so the data can't be changed afterwards without detection."*

Part 11 — Conclusion

STEP 40 — Close

SAY

"To summarise: AetherOS shows the classic OS mechanisms — scheduling on two cores, paging, deadlock detection and avoidance, and journaling — in a way you can see and test. SpillGuard applies the same memory-management idea to a real, modern problem: GPU memory silently spilling into system RAM. I built a live detector, used Banker's algorithm for real admission control, and proved with controlled experiments that crossing VRAM capacity by 256 MB makes a local AI model 2.4 times slower, that memory counters alone can't distinguish harmful pressure, and that pausing the other program fixes it while keeping its memory. Limitations: one laptop, one GPU, two trials per level, and the churn mechanism is still a hypothesis. Next I want to verify it with ETW tracing and test more GPUs. Thank you, Ma'am."

If something goes wrong

Problem	What to do
SpillGuard says "Waiting for the agent…" or shows only Simulated	You opened the page without the token. Close the black window, double-click `START_DEMO.bat` again and use the tab it opens. If time is short, use **Replay → Watch the demo spill** — it works without the agent.
A window opened "behind" another one	Close the maximized window with red, or minimize it with green. Menu logo → **Reset Windows Layout** tidies everything.
Clicking a menu item seems to do nothing	The window probably opened under a maximized window — close/minimize the top one.
Task Manager timelines are all grey	All processes finished (or the clock is paused). Dispatch new processes and/or press Resume Clock.
"Value must be less than or equal to 50"	Burst can be at most 50.
Memory Map doesn't fill to 32/32	Processes finished before you looked. Pause Clock, dispatch 9 with burst 50, Resume, open the Memory Map right away.
"Will it fit?" says UNSAFE because of commit	System RAM is nearly full. Explain it as the type C (host RAM) guard, or close apps and try again.
Red banner "RAM_PRESSURE" or events of type C	Same reason — RAM is nearly full. Close browsers/apps.
Want a fresh start	Press **F5**: the page reloads and the simulated kernel restarts from tick 0.
Black START_DEMO window closed by mistake	Double-click `START_DEMO.bat` again.

Shortcut map — where everything is

Thing	How to open
Task Manager (scheduler)	Desktop **Dash's Journal**
Memory Map	Diagnostic → Virtual Memory Map
Deadlock Engine + Banker's	Diagnostic → Open Deadlock Engine (or Edit → Simulate Deadlock)
Kernel Test Suite	Diagnostic → Run Kernel Test Suite (or logo menu → Diagnostic Test Runner)
KernelQuest challenges	Diagnostic → KernelQuest Challenges (deadlock, starvation/MLFQ, "Who is the culprit?", "Will this model fit?")
Kernel event log	Logo menu → Kernel Audit Journal
Finder (file system, power cut, fsck)	Desktop **archive** (or dock Finder)
Terminal	Dock **>_** (or File → Open Terminal CLI)
eBPF Studio	Desktop **eBPF Studio**
SpillGuard	Desktop **SpillGuard** (or logo menu → SpillGuard)
Pause / step the CPU clock	Kernel → Pause Clock / Step Single Tick
Protection fault	Terminal `gpf` (shows the dump) — Edit → Trigger Ring 3 Fault does it silently

Full explanations of every term are in the companion document **AetherOS_SpillGuard_Complete_Guide.pdf** (sections 17–19: viva questions, glossary and numbers cheat-sheet).